

	ESPECIFICACIÓN DE DISEÑO DE SOFTWARE (SDS)	
Universidad Piloto de Colombia	PROYECTO: DASHKPI'S	Grupo:Print(Null);
		Ciclo: 1

Objetivo del documento.....	2
Audiencia del documento.....	2
Acrónimos.....	3
Descripción del problema.....	3
Objetivos de la aplicación.....	5
Restricciones.....	5
Arquitectura de software.....	6
Estilo de arquitectura.....	9
Vistas de arquitectura.....	9
Contexto.....	10
Funcional.....	14
Información.....	15
Despliegue.....	16
Desarrollo.....	17
Concurrencia.....	18
Operacional.....	19
Diseño detallado.....	26
Especificación de la interfaz de usuario.....	26
Modelo de datos entidad relación.....	29
Modelo de clases.....	30

	ESPECIFICACIÓN DE DISEÑO DE SOFTWARE (SDS)		
Universidad Piloto de Colombia	PROYECTO: DASHKPI'S	Grupo:Print(Null);	
		Ciclo:	1

Objetivo del documento

Definir y documentar la arquitectura y el diseño del sistema **DashKPI**, proporcionando una guía clara y estructurada para el desarrollo, implementación y validación del software, asegurando que cumpla con los requerimientos funcionales, no funcionales y los indicadores clave de desempeño (KPIs) establecidos.

1. Definir la arquitectura y diseño del sistema DashKPI, estableciendo cómo se implementarán los requerimientos funcionales y no funcionales.
2. Servir como guía de referencia para el equipo de desarrollo durante la construcción del software.
3. Facilitar la comunicación entre líderes de rol, analistas, desarrolladores, testers y clientes, asegurando una visión común del sistema.
4. Garantizar la trazabilidad entre los requerimientos del proyecto y las decisiones de diseño.
5. Reducir riesgos técnicos, documentando las decisiones clave en cuanto a arquitectura, interfaz de usuario, base de datos y seguridad.
6. Apoyar la validación y verificación del sistema, asegurando que el producto final cumpla con los objetivos de negocio y los indicadores (KPIs) definidos.

Audiencia del documento

Equipo de desarrollo: para guiar la implementación técnica de DashKPI según las decisiones de diseño.

Líderes de rol (planeación, proyecto, desarrollo, calidad, soporte, arquitectura): para entender cómo la arquitectura soporta sus actividades dentro del ciclo de vida del software.

	ESPECIFICACIÓN DE DISEÑO DE SOFTWARE (SDS)		
Universidad Piloto de Colombia	PROYECTO: DASHKPI'S	Grupo:Print(Null);	
		Ciclo:	1

Clientes y usuarios finales: para conocer de manera general el alcance del sistema y cómo se integran los KPIs con la gestión de tareas.

Profesores y evaluadores académicos: para validar la consistencia entre requerimientos, arquitectura y diseño del sistema.

Acrónimos

SDS: *Software Design Specification* (Especificación de Diseño de Software).

KPI: *Key Performance Indicator* (Indicador Clave de Desempeño).

PM: *Project Manager* (Líder de Proyecto).

DB: *Database* (Base de Datos).

UI: *User Interface* (Interfaz de Usuario).

API: *Application Programming Interface* (Interfaz de Programación de Aplicaciones).

TSP: *Team Software Process* (Proceso de Software en Equipo).

Descripción del problema

En muchos entornos de desarrollo de software, la gestión y monitoreo del rendimiento de los proyectos y tareas a través de indicadores clave de desempeño (KPIs) es fundamental para asegurar la calidad, eficiencia y éxito de las iniciativas. Sin embargo, los equipos a menudo

	ESPECIFICACIÓN DE DISEÑO DE SOFTWARE (SDS)		
Universidad Piloto de Colombia	PROYECTO: DASHKPI'S	Grupo:Print(Null);	
		Ciclo:	1

enfrentan dificultades para obtener una visión clara y precisa del progreso de sus proyectos debido a la falta de herramientas integradas que conecten las tareas, el tiempo invertido, los KPIs y los roles de los diferentes equipos involucrados.

El proyecto DashKPI busca abordar esta problemática creando una plataforma integral para la gestión de KPIs, que permita a los equipos y líderes de proyecto monitorear y gestionar sus actividades de manera efectiva. Sin embargo, actualmente se presentan los siguientes desafíos:

1. Desconexión entre tareas y KPIs: Los KPIs no están directamente vinculados a las tareas específicas del proyecto, lo que dificulta la visualización del impacto de cada actividad en el rendimiento general del proyecto. Esto crea dificultades para tomar decisiones informadas y para ajustar los esfuerzos de los equipos en tiempo real.
2. Falta de visibilidad del rendimiento en tiempo real: Sin una herramienta centralizada para la visualización de los KPIs y el progreso de las tareas, los equipos y roles clave (Líder de Calidad, Líder de Desarrollo, etc.) carecen de información precisa sobre el desempeño, lo que retrasa la toma de decisiones críticas y afecta la calidad general del proyecto.
3. Gestión ineficiente de tareas y recursos: Los procesos manuales y desorganizados de seguimiento de tareas no permiten una asignación eficiente de recursos ni un registro claro del tiempo y esfuerzo invertido, lo que puede generar cuellos de botella y afectar la entrega puntual de los proyectos.
4. Falta de control en la calidad del software: Los defectos en el software no siempre se registran ni se gestionan de manera adecuada, lo que dificulta la corrección y mejora continua del producto. Sin un sistema eficiente de gestión de defectos, los equipos pueden pasar por alto problemas críticos, lo que impacta negativamente en la calidad y en la satisfacción del cliente.
5. Desafíos en la generación de reportes y análisis: La ausencia de reportes automáticos y detallados sobre los KPIs y las tareas dificulta la evaluación precisa del rendimiento del

	ESPECIFICACIÓN DE DISEÑO DE SOFTWARE (SDS)		
Universidad Piloto de Colombia	PROYECTO: DASHKPI'S	Grupo:Print(Null);	
		Ciclo:	1

proyecto. Esto impide que los equipos tomen decisiones basadas en datos en tiempo real, afectando la capacidad para optimizar procesos y mejorar la eficiencia.

Objetivos de la aplicación

Objetivo General:

Desarrollar e implementar una plataforma integral de gestión de KPIs, tareas y defectos en el proyecto DashKPI, que permita monitorear el rendimiento en tiempo real, optimizar la asignación de recursos y garantizar la calidad continua del software.

Objetivos Específicos:

1. Integrar KPIs con tareas para permitir el seguimiento automático del rendimiento del proyecto y facilitar la toma de decisiones basada en datos.
2. Gestionar y hacer seguimiento de defectos de manera eficiente, asegurando que cada defecto se registre, asigne, y cierre adecuadamente, mejorando la calidad del software.
3. Automatizar la generación de reportes sobre el progreso de las tareas y el estado de los KPIs, proporcionando visibilidad y facilitando la toma de decisiones estratégicas.

Restricciones

Alcance y negocio

Todas las tareas deben estar asociadas a un KPI o justificar su ausencia. Los roles (PM, líderes, colaboradores) están estrictamente definidos, y los defectos deben seguir un flujo formal desde su apertura hasta el cierre validado.

Datos y consistencia

La información se gestiona en MySQL con integridad referencial obligatoria. Los cambios quedan registrados en un historial inmutable. Catálogos como severidad, prioridad y estado están controlados para

	ESPECIFICACIÓN DE DISEÑO DE SOFTWARE (SDS)		
Universidad Piloto de Colombia	PROYECTO: DASHKPI'S	Grupo:Print(Null);	
		Ciclo:	1

garantizar uniformidad.

Seguridad

El sistema requiere autenticación segura y autorización por roles. Los datos sensibles deben almacenarse cifrados, todo tráfico será bajo HTTPS y las acciones críticas deben quedar registradas en auditoría.

Arquitectura tecnológica

El frontend (React) solo consume datos mediante el backend (Django), nunca directamente de la base de datos. Los endpoints deben ser eficientes, paginados y con filtros. Notificaciones y reportes se entregan de manera automática y consistente.

Calidad de servicio

El tiempo de respuesta en operaciones críticas no debe superar los 400 ms en promedio. La disponibilidad mínima será de 99.5% mensual, con capacidad de escalar horizontalmente.

Interfaz de usuario

Las vistas deben mantener coherencia visual, accesibilidad (WCAG 2.1 AA) y mensajes claros ante errores. Cada interacción debe estar ligada a una tarea y su KPI asociado.

Proceso y documentación

Los diagramas UML (casos de uso, componentes, despliegue, ER, secuencia) deben ser consistentes entre sí. Una historia se considera finalizada solo si incluye código, pruebas, documentación y métricas de KPI actualizadas.

Arquitectura de software

La arquitectura de software de este proyecto constituye la base conceptual y estructural sobre la cual se desarrollará la solución orientada a la gestión de tareas y el cálculo en tiempo real de sus indicadores clave de desempeño (KPIs), involucrando a todos los actores definidos (PM, Colaborador, Stakeholder y Administrador). Esta arquitectura se enmarca en una transición entre las fases de diseño y desarrollo, lo que implica que cada una de sus vistas y componentes no solo

	ESPECIFICACIÓN DE DISEÑO DE SOFTWARE (SDS)		
Universidad Piloto de Colombia	PROYECTO: DASHKPI'S	Grupo:Print(Null);	
		Ciclo:	1

debe ser consistente a nivel técnico, sino también trazable respecto a los requerimientos funcionales y de calidad previamente definidos.

El estilo de arquitectura adoptado corresponde a un modelo por capas complementado con un enfoque orientado a eventos, en el cual el Front-end (React) se encarga de la interacción con los usuarios, el Back-end (Django) implementa la lógica de negocio y el motor de KPIs, y la capa de Persistencia (MySQL y Redis) garantiza la integridad, disponibilidad y consistencia de la información. Este estilo permite asegurar separación de responsabilidades, escalabilidad, mantenibilidad y soporte para operaciones en tiempo real.

Para cubrir de manera integral el diseño arquitectónico, se desarrollarán las siguientes vistas UML:

- Vista de Contexto: delimita el sistema y muestra la relación con actores e integraciones externas.
- Vista Funcional: describe los módulos principales (tareas, usuarios, KPIs, reportes) y sus interacciones.
- Vista de Información: representa la estructura de datos, entidades clave y su persistencia en la base de datos.
- Vista de Despliegue: define la infraestructura física y lógica para ejecutar los componentes (nodos, servidores, servicios externos).
- Vista de Desarrollo: organiza el código en paquetes, frameworks y librerías según las responsabilidades.
- Vista de Concurrencia: explica cómo se gestionan procesos paralelos y asincronía (ejecución de KPIs, notificaciones en tiempo real).

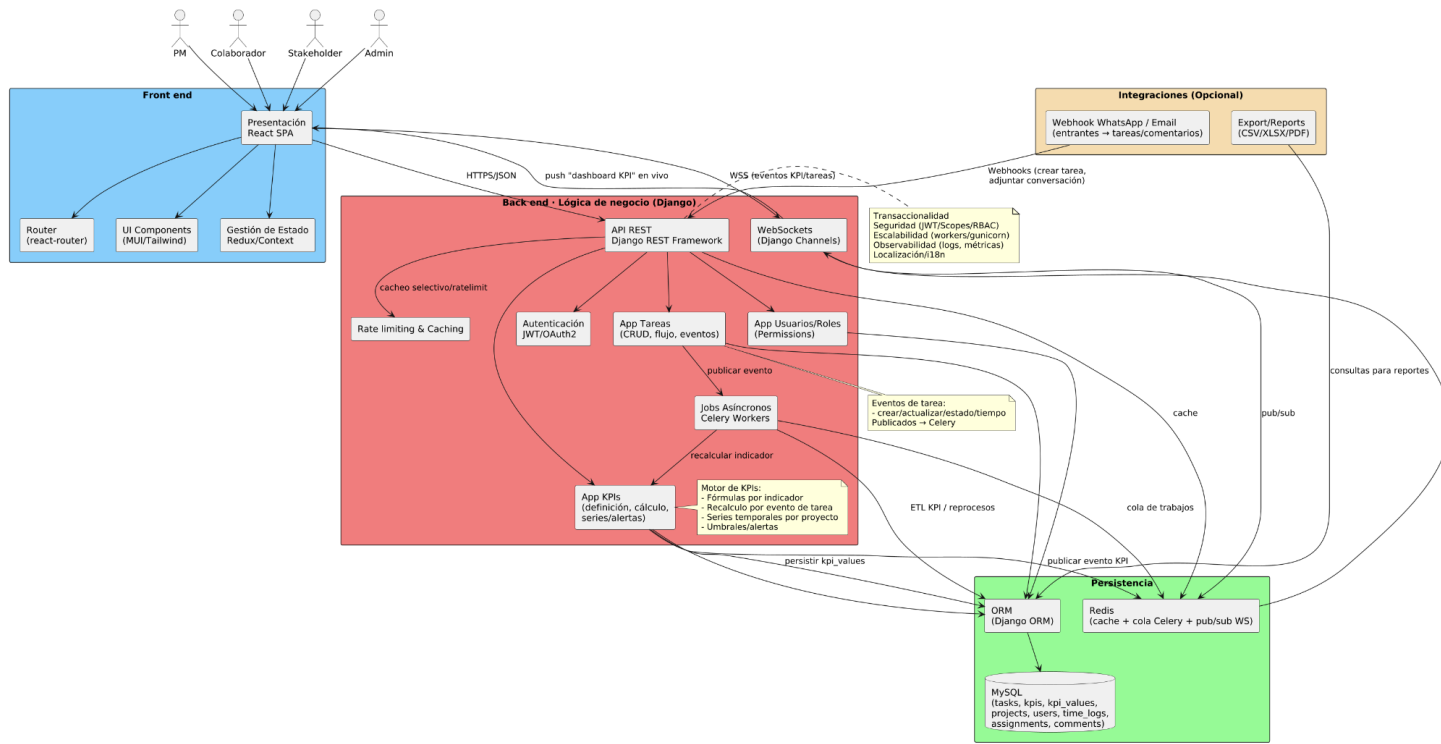
	ESPECIFICACIÓN DE DISEÑO DE SOFTWARE (SDS)		
Universidad Piloto de Colombia	PROYECTO: DASHKPI'S	Grupo:Print(Null);	
		Ciclo:	1

- Vista Operacional: contempla la administración, monitoreo y mantenimiento del sistema en un entorno productivo.

En conjunto, estas vistas constituyen un marco de referencia lógico y consistente para la implementación, asegurando que cada decisión de diseño apoye directamente la consecución de los objetivos del sistema: facilitar la gestión colaborativa de tareas y proporcionar información clara y confiable mediante KPIs en tiempo real.

	ESPECIFICACIÓN DE DISEÑO DE SOFTWARE (SDS)	
Universidad Piloto de Colombia	PROYECTO: DASHKPI'S	Grupo:Print(Null);
		Ciclo: 1

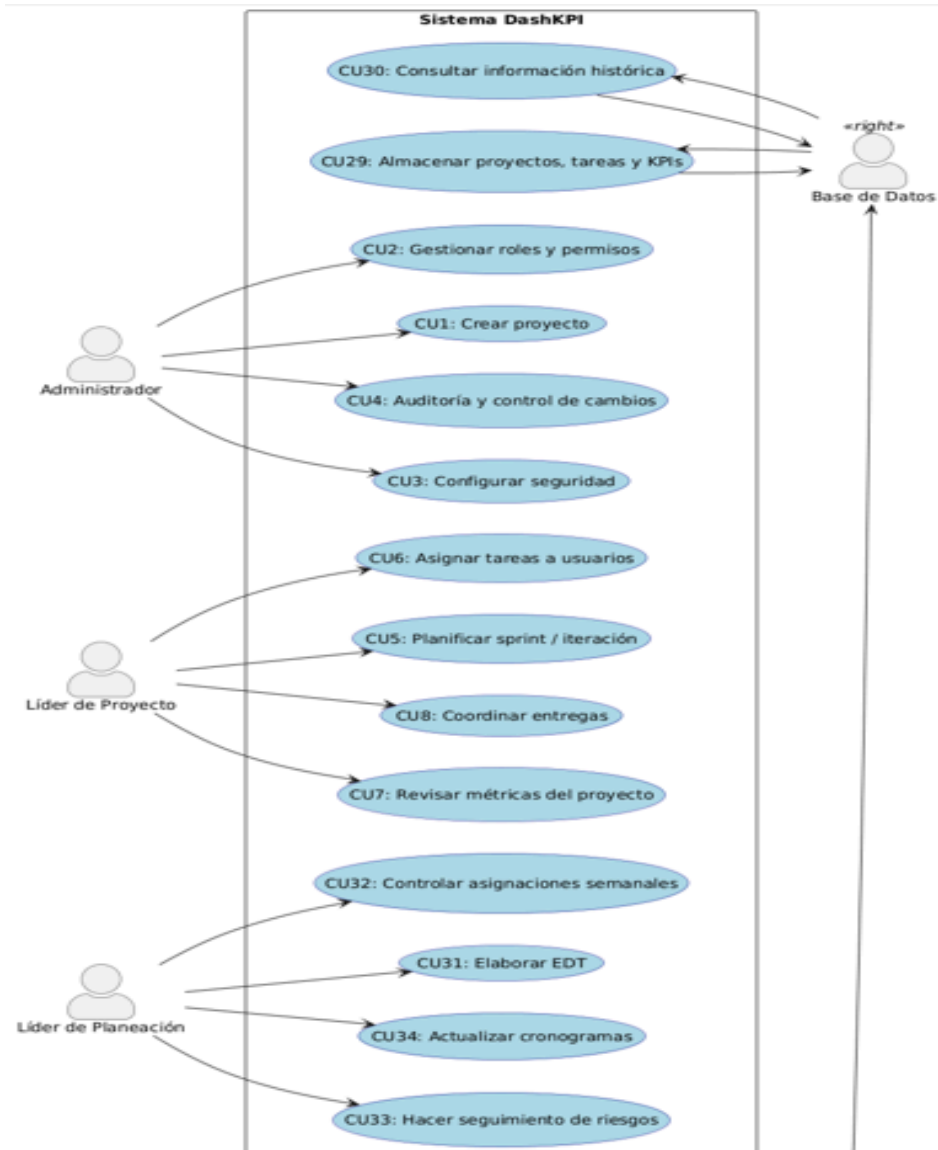
Estilo de arquitectura



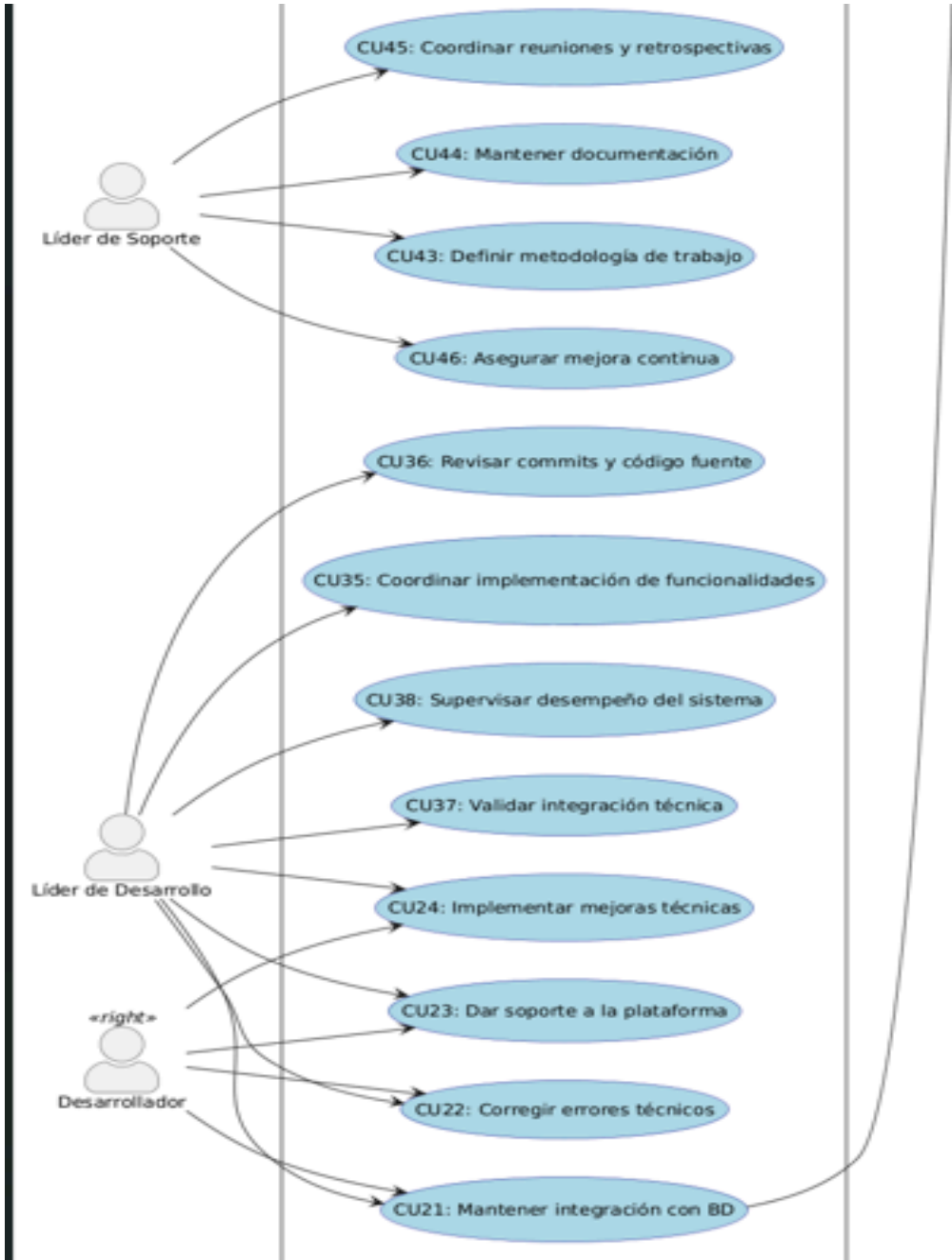
Vistas de arquitectura

	ESPECIFICACIÓN DE DISEÑO DE SOFTWARE (SDS)		
Universidad Piloto de Colombia	PROYECTO: DASHKPI'S	Grupo:Print(Null);	
		Ciclo:	1

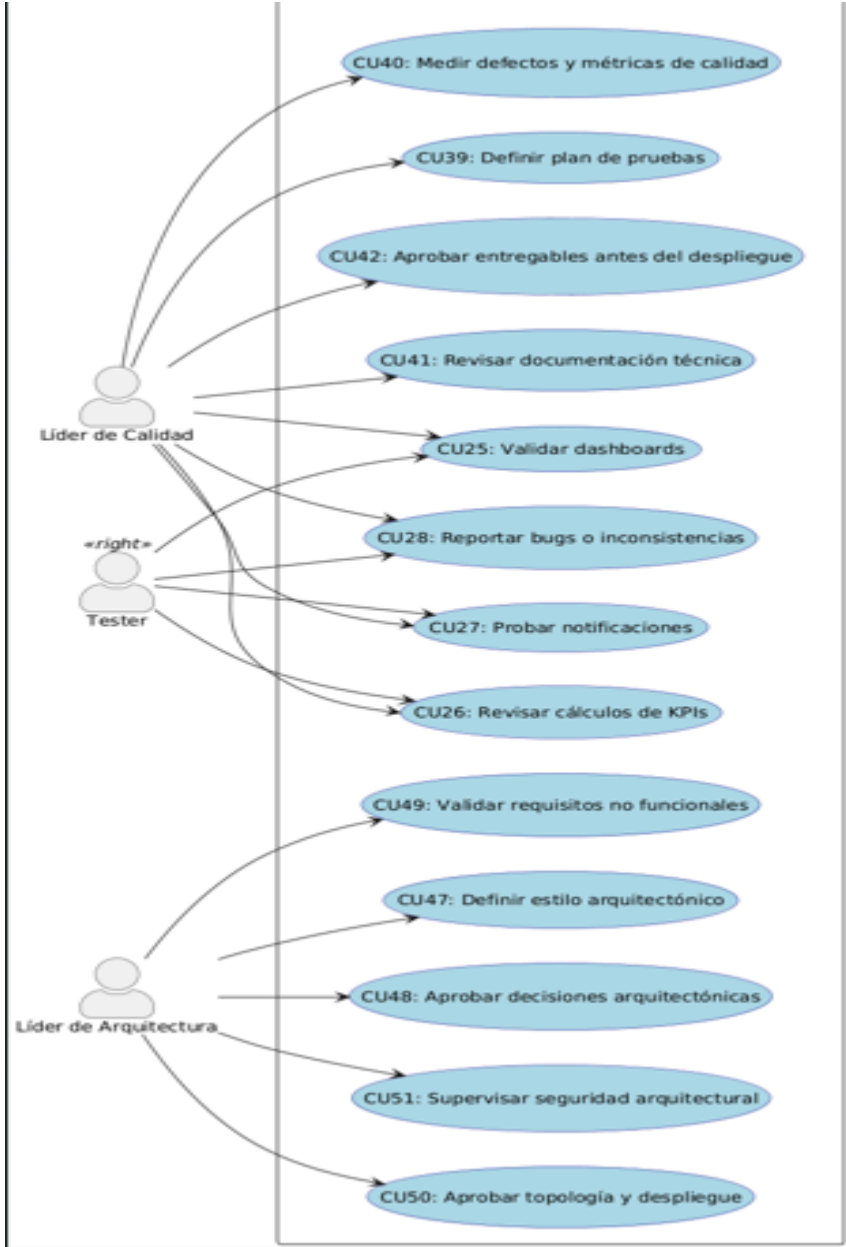
Contexto



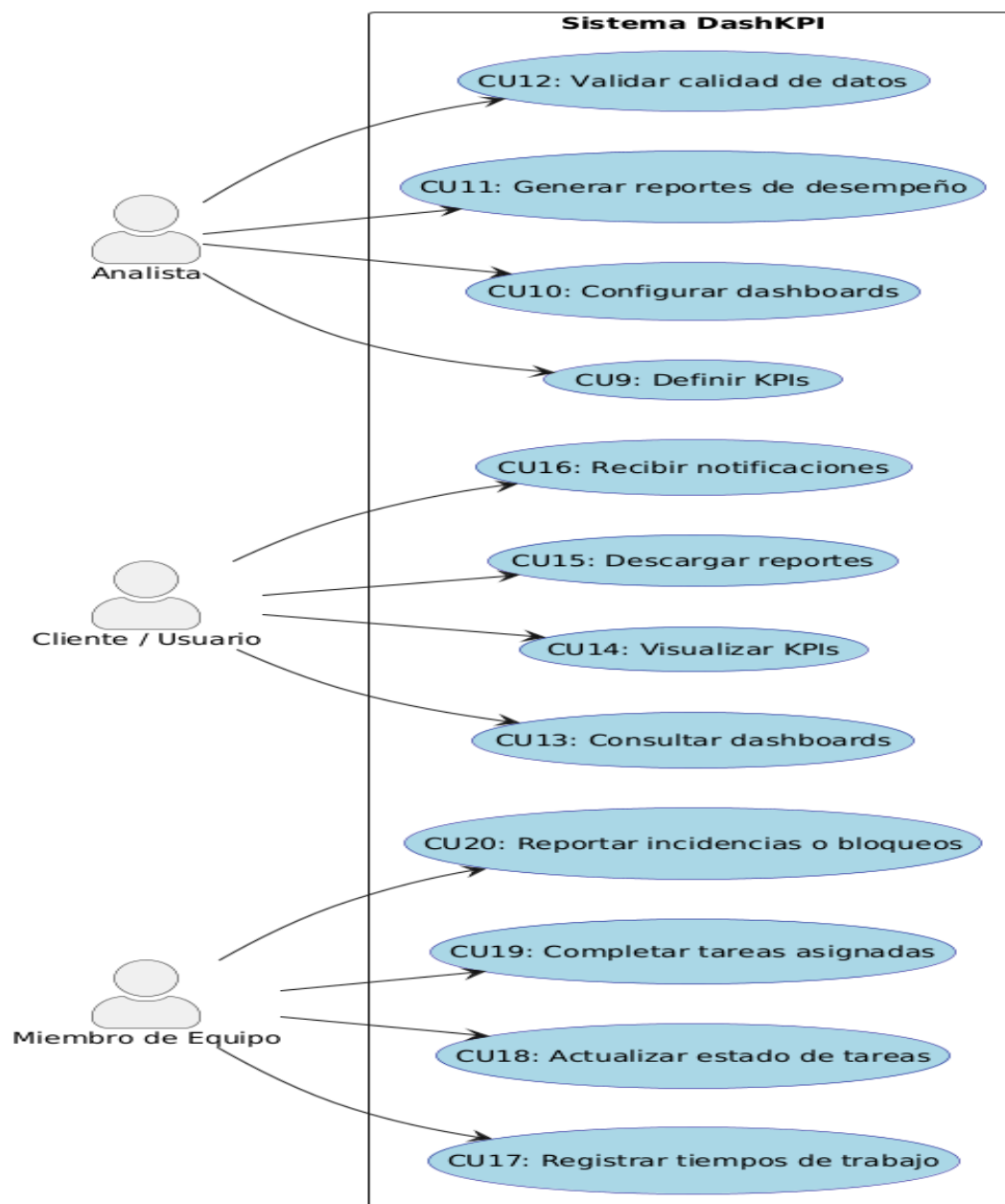
	ESPECIFICACIÓN DE DISEÑO DE SOFTWARE (SDS)	
Universidad Piloto de Colombia	PROYECTO: DASHKPI'S	Grupo:Print(Null);
		Ciclo: 1



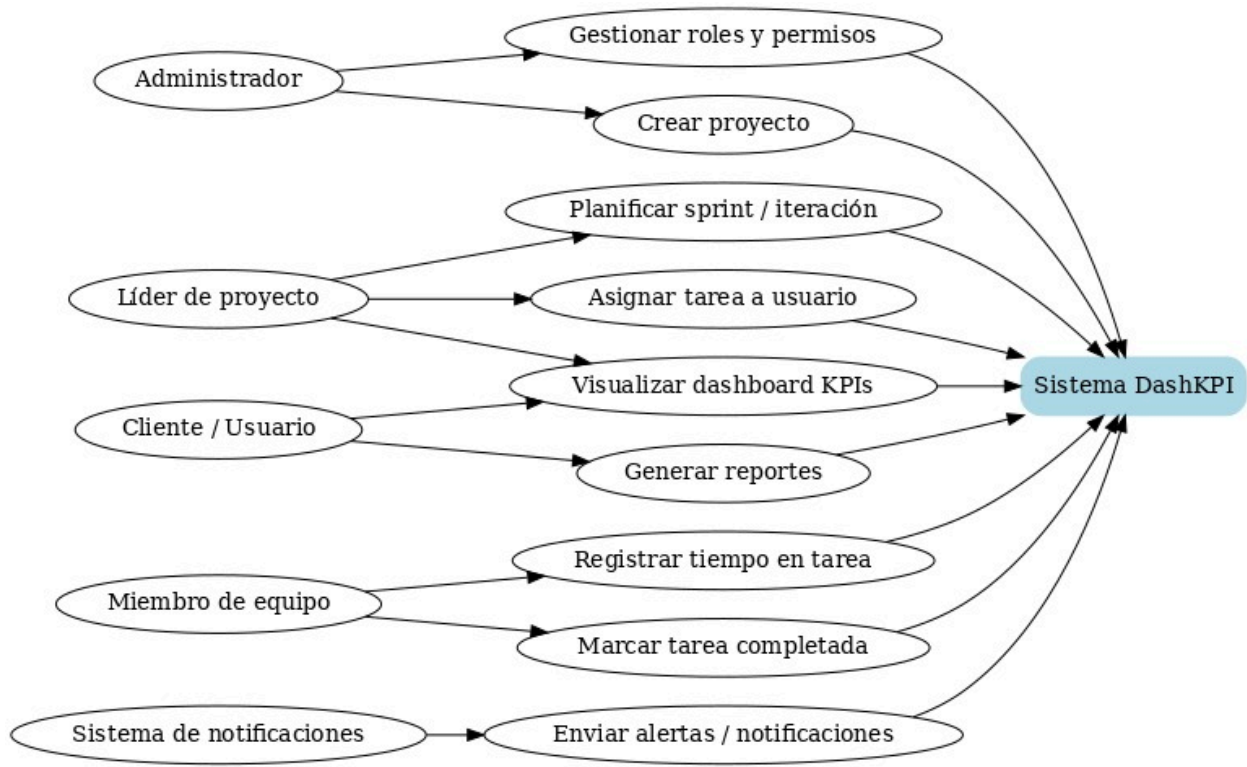
	ESPECIFICACIÓN DE DISEÑO DE SOFTWARE (SDS)	
Universidad Piloto de Colombia	PROYECTO: DASHKPI'S	Grupo:Print(Null);
		Ciclo: 1



	ESPECIFICACIÓN DE DISEÑO DE SOFTWARE (SDS)		
Universidad Piloto de Colombia	PROYECTO: DASHKPI'S	Grupo:Print(Null);	
		Ciclo:	1

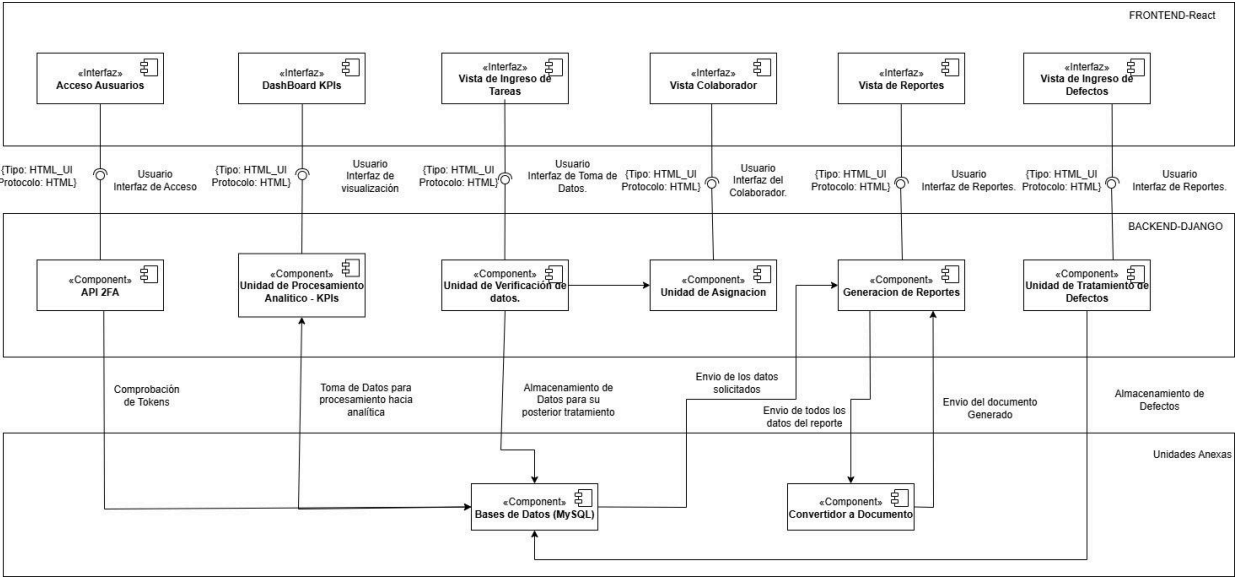


	ESPECIFICACIÓN DE DISEÑO DE SOFTWARE (SDS)		
Universidad Piloto de Colombia	PROYECTO: DASHKPI'S	Grupo:Print(Null);	
		Ciclo:	1



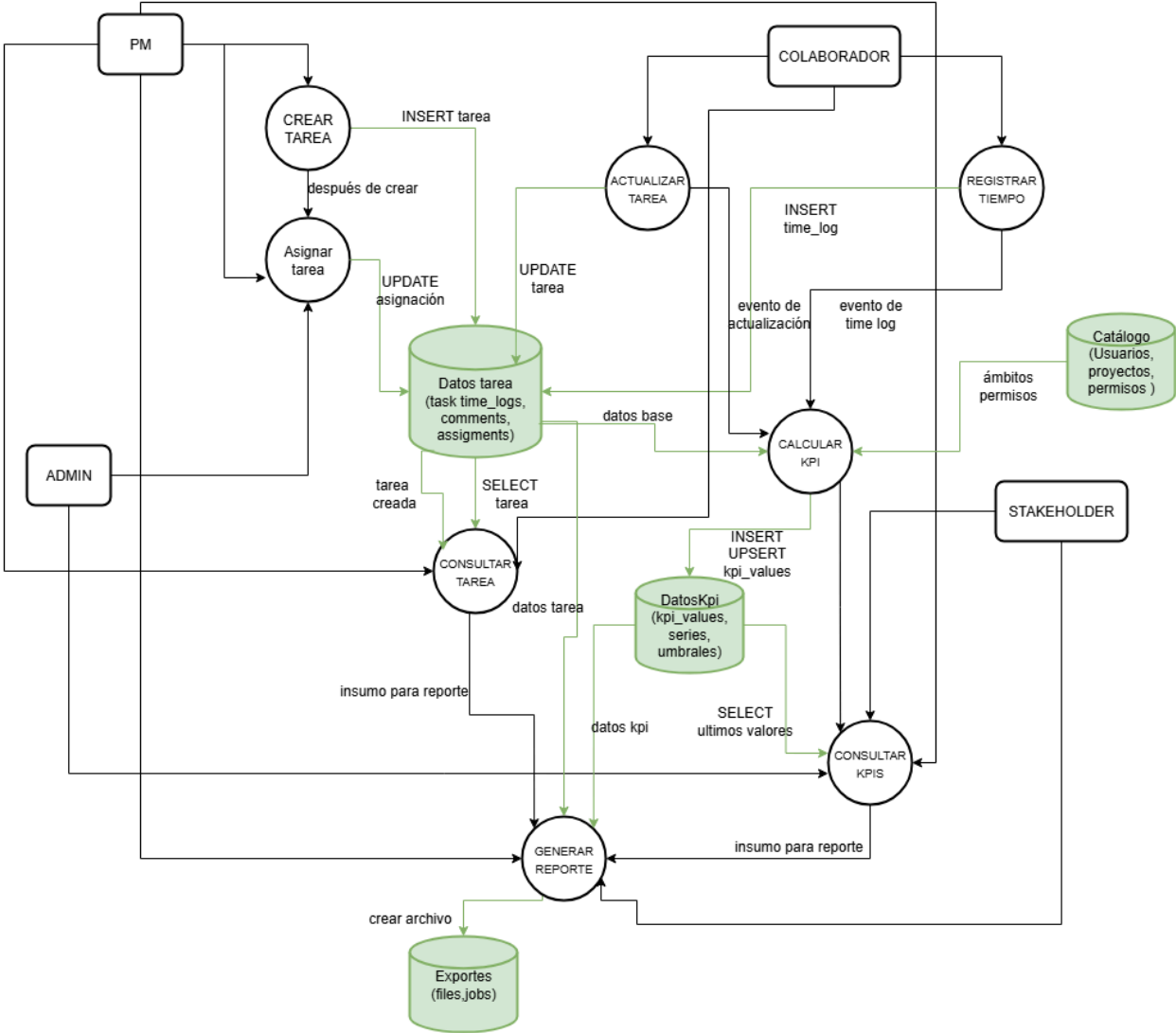
	ESPECIFICACIÓN DE DISEÑO DE SOFTWARE (SDS)	
Universidad Piloto de Colombia	PROYECTO: DASHKPI'S	Grupo:Print(Null);
		Ciclo: 1

Funcional



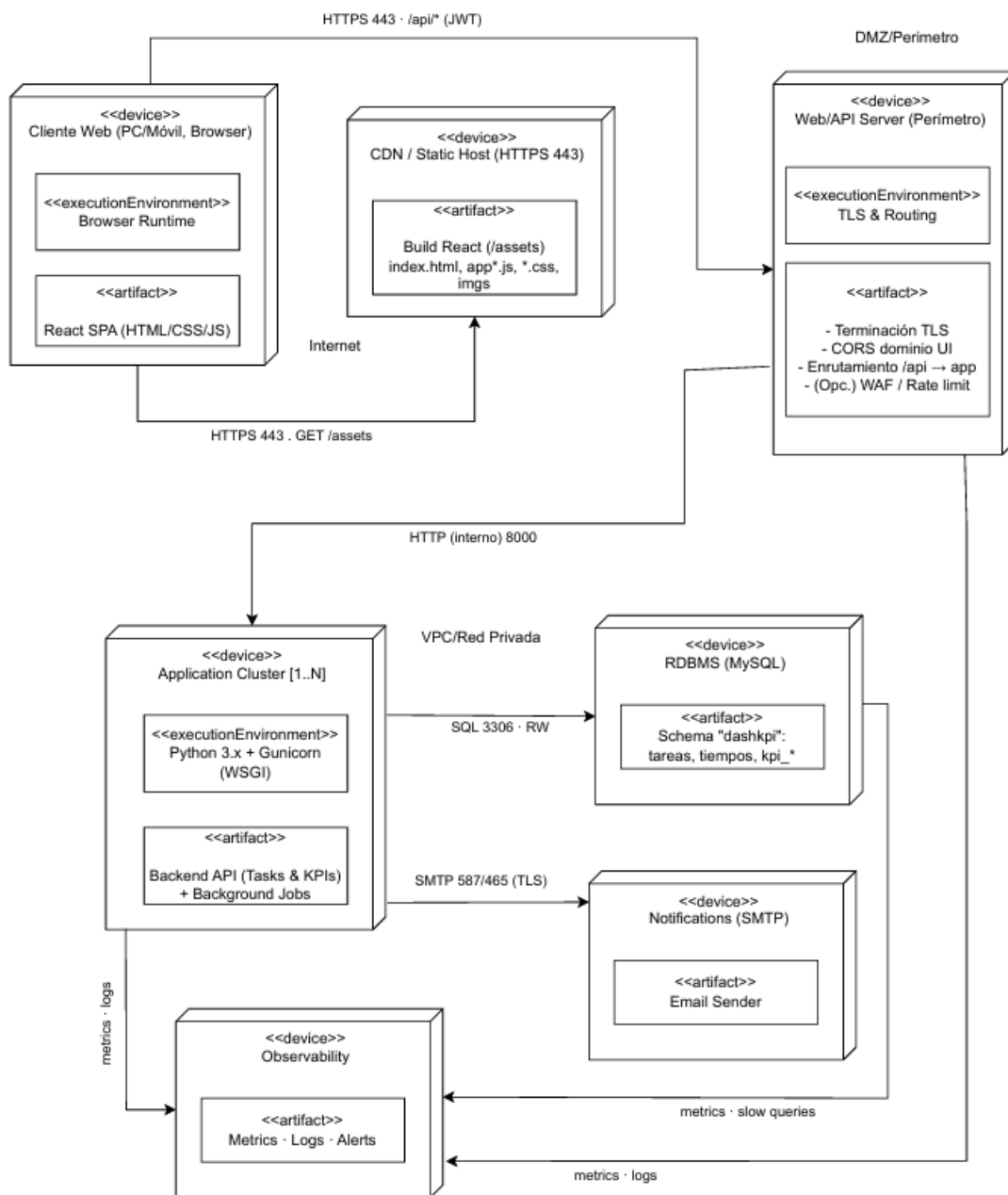
	ESPECIFICACIÓN DE DISEÑO DE SOFTWARE (SDS)	
Universidad Piloto de Colombia	PROYECTO: DASHKPI'S	Grupo:Print(Null);
		Ciclo: 1

Información



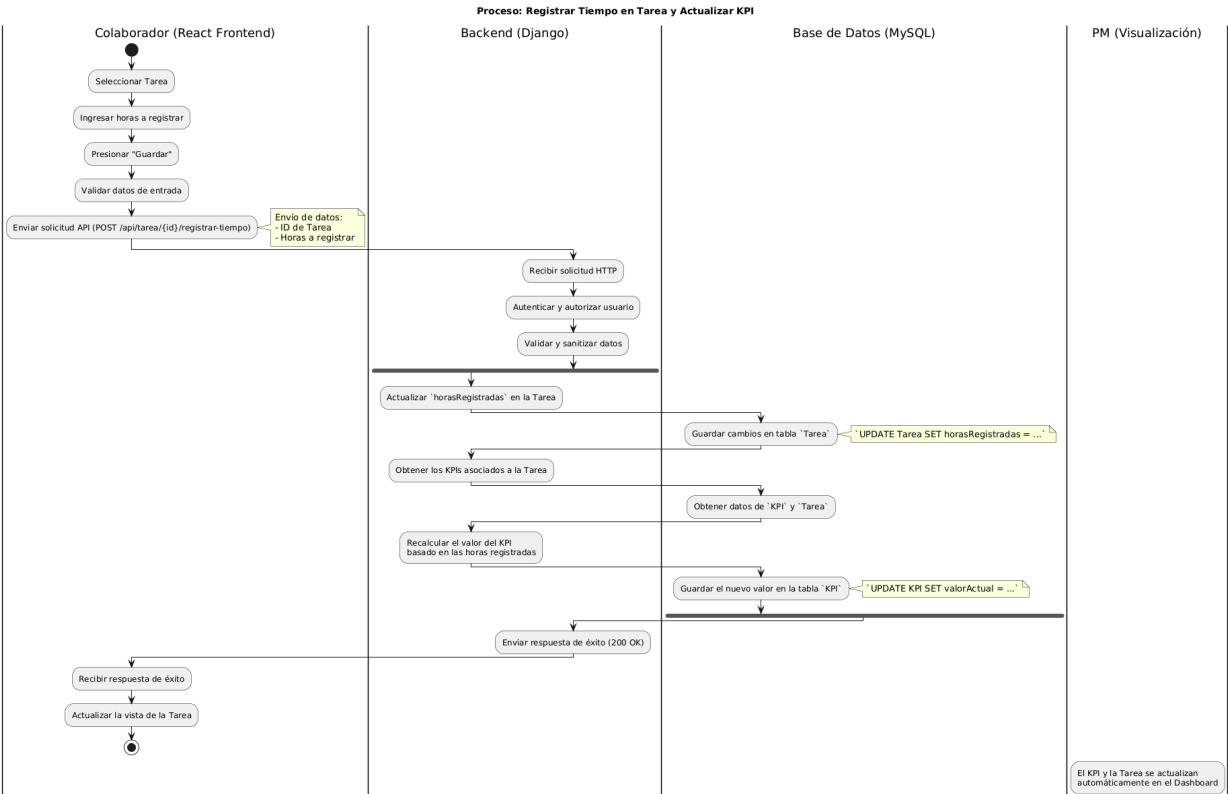
	ESPECIFICACIÓN DE DISEÑO DE SOFTWARE (SDS)	
Universidad Piloto de Colombia	PROYECTO: DASHKPI'S	Grupo:Print(Null);
		Ciclo: 1

Despliegue



	ESPECIFICACIÓN DE DISEÑO DE SOFTWARE (SDS)	
Universidad Piloto de Colombia	PROYECTO: DASHKPI'S	Grupo:Print(Null);
		Ciclo: 1

Concurrencia



	ESPECIFICACIÓN DE DISEÑO DE SOFTWARE (SDS)		
Universidad Piloto de Colombia	PROYECTO: DASHKPI'S	Grupo:Print(Null);	
		Ciclo:	1

Operacional

1. SLOs sugeridos (servicio)

- Disponibilidad API $\geq 99.5\%$ mensual.
- Latencia p95 (GET /tasks, GET /kpi-values) ≤ 400 ms.
- Error rate (5xx) $\leq 0.5\%$.
- Tiempo de recálculo KPI ≤ 2 min desde el evento de tarea.
- Tasa de entrega WS (eventos KPI) $\geq 99\%$ en < 3 s.

2. Componentes operados

- Frontend: React (build estático en Nginx/CDN).
- Back-end web: Django (Gunicorn/ASGI + Nginx o Daphne/Uvicorn para Channels).
- Workers: Celery + Celery Beat (colas en Redis).
- Mensajería/Cache: Redis (pub/sub, cache, cola).
- Base de datos: MySQL (datos de tareas/KPIs).
- Observabilidad: logs (app/web), métricas (Prometheus/Grafana o equivalente), traces opcional.

Tareas operativas rutinarias

	ESPECIFICACIÓN DE DISEÑO DE SOFTWARE (SDS)		
Universidad Piloto de Colombia	PROYECTO: DASHKPI'S	Grupo:Print(Null);	
		Ciclo:	1

3. Diarias

- Verificar estado de servicios: Nginx, Django web, Channels, Celery, Beat, Redis, MySQL.
- Revisar colas Celery (profundidad y tiempos de espera).
- Chequear errores 5xx y latencias p95 de endpoints críticos.
- Confirmar backups de MySQL (que el archivo existe y su tamaño > 0).
- Revisar disco (> 20% libre) y certificados TLS (vigencia).

4. Semanales

- Analizar slow queries MySQL; agregar índices si aplica.
- Revisar ratio de cache (Redis hit-rate) y memoria.
- Validar kpi_values (consistencia vs tareas de la semana).
- Rotación de logs y limpieza de artefactos antiguos.
- Probar restore parcial en entorno de pruebas.

5. Mensuales

- Rotar secretos (JWT secret, credenciales DB, claves Webhook).
- Prueba DR: restaurar un backup completo en ambiente aislado y validar login, consulta tareas/KPIs.
- Revisar política de retención (logs, exportes).

	ESPECIFICACIÓN DE DISEÑO DE SOFTWARE (SDS)		
Universidad Piloto de Colombia	PROYECTO: DASHKPI'S	Grupo:Print(Null);	
		Ciclo:	1

6. Despliegue (externo) – flujo recomendado

1. Compilar Frontend: `npm ci && npm run build` → publicar en Nginx/CDN.
2. Empaquetar Backend: build de imagen/artefacto.
3. Ventana de despliegue: anunciar al equipo/PM.
4. Paralelo o rolling:
 - Aplicar migraciones: `python manage.py migrate`.
 - Ejecutar `collectstatic` si aplica.
 - Reiniciar workers Celery y Beat (para tomar nuevos tasks).
 - Reiniciar web (Gunicorn/Daphne).
5. Smoke tests: `/health`, login, GET `/tasks`, `/kpi-values`.
6. Monitoreo cercano 30–60 min (errores/latencia/colas).
7. Rollback (si falla): volver a la imagen anterior + `migrate` reversibles (si se versiona).

Sugerido: blue/green para web (cambiar tráfico al nuevo pool cuando smoke OK).

7.Backups y restauración

- MySQL (diario, nocturno):
 - `mysqldump --single-transaction --routines --triggers db > backup-YYYYMMDD.sql.gz`

	ESPECIFICACIÓN DE DISEÑO DE SOFTWARE (SDS)		
Universidad Piloto de Colombia	PROYECTO: DASHKPI'S	Grupo:Print(Null);	
		Ciclo:	1

- Guardar en almacenamiento externo (S3, bucket on-prem), retención 30 días.
- Redis: normalmente no crítico para backup (cache/cola), salvo si usas RDB/AOF con retención.
- Restore (pruebas semanales):
 - Crear DB nueva → `mysql db < backup.sql`
 - Reconfigurar app (var de entorno) y validar login, tareas y KPIs de una fecha.

8. Monitoreo y alertas (mínimo)

- API: disponibilidad, latencia p95, 5xx.
- Celery: profundidad de cola, tasks fallidas, *task runtime* p95.
- WebSockets: conexiones activas, eventos por minuto y *delivery lag*.
- MySQL: CPU/IO, conexiones, *slow queries*, locks.
- Redis: memoria usada, evictions, hit-rate, lag de pub/sub.
- Disco y RAM: umbrales 80/90%.
- Certificados TLS: caducidad (<15 días → alerta).

9. Playbooks de incidentes

	ESPECIFICACIÓN DE DISEÑO DE SOFTWARE (SDS)		
Universidad Piloto de Colombia	PROYECTO: DASHKPI'S	Grupo:Print(Null);	
		Ciclo:	1

1. Cola Celery creciendo

- Confirmar que Workers y Beat están UP.
- Escalar workers (réplicas o concurrency).
- Identificar tasks lentas (logs/metrics) → optimizar consultas/índices.
- Si Redis saturado: aumentar memoria, limpiar claves viejas, revisar TTLs.

2. KPIs no se actualizan

- Verificar eventos de tarea llegan a la cola.
- Revisar traza de task `recalculate_kpi`.
- Validar acceso a MySQL (locks/slow).
- Confirmar pub/sub Redis y Channels (WS) activos.

3. Error rate 5xx alto

- Inspeccionar últimos despliegues.
- Revisar logs de Django/DRF y dependencias externas (webhooks).
- Rollback si necesario; abrir postmortem.

4. Latencia alta

- Chequear slow queries y índices.

	ESPECIFICACIÓN DE DISEÑO DE SOFTWARE (SDS)		
Universidad Piloto de Colombia	PROYECTO: DASHKPI'S	Grupo:Print(Null);	
		Ciclo:	1

- Aumentar réplicas web y cache selectivo en endpoints calientes.
- Verificar CDN para estáticos.

5. DB sin espacio / disco lleno

- Limpiar logs/exports antiguos.
- Extender volumen o comprimir backups.
- Reiniciar servicios que quedaron en fallo por E/S.

10. Seguridad y acceso

- RBAC para consolas (monitoring, DB, servidores).
- Rotación de secretos mensual (o tras cambios de personal).
- Principio de mínimo privilegio para cuentas de servicio (Celery, CI/CD).
- Webhooks firmados/verificados.

11. Gobierno de datos

- Retención: `time_logs` (2 años), `kpi_values` (1 año agregados / 90 días crudos).
- Privacidad: ofuscar datos personales en entornos de prueba.
- Integridad: migraciones versionadas, seeds controlados.

	ESPECIFICACIÓN DE DISEÑO DE SOFTWARE (SDS)		
Universidad Piloto de Colombia	PROYECTO: DASHKPI'S	Grupo:Print(Null);	
		Ciclo:	1

11. RACI (operación)

- DevOps/SRE: despliegues, monitoreo, backups, DR, escalado.
- DBA: tuning MySQL, índices, restore.
- Soporte: atención de alertas de primer nivel, comunicación.
- Líder técnico/Arquitecto: cambios de arquitectura, revisión postmortem.
- PM: ventanas de cambio, comunicación con stakeholders.

	ESPECIFICACIÓN DE DISEÑO DE SOFTWARE (SDS)		
Universidad Piloto de Colombia	PROYECTO: DASHKPI'S	Grupo:Print(Null);	
		Ciclo:	1

Diseño detallado

Especificación de la interfaz de usuario

Start your journey

Sign Up to InsideBox

E-mail

example@email.com

Password

.....

Sign Up

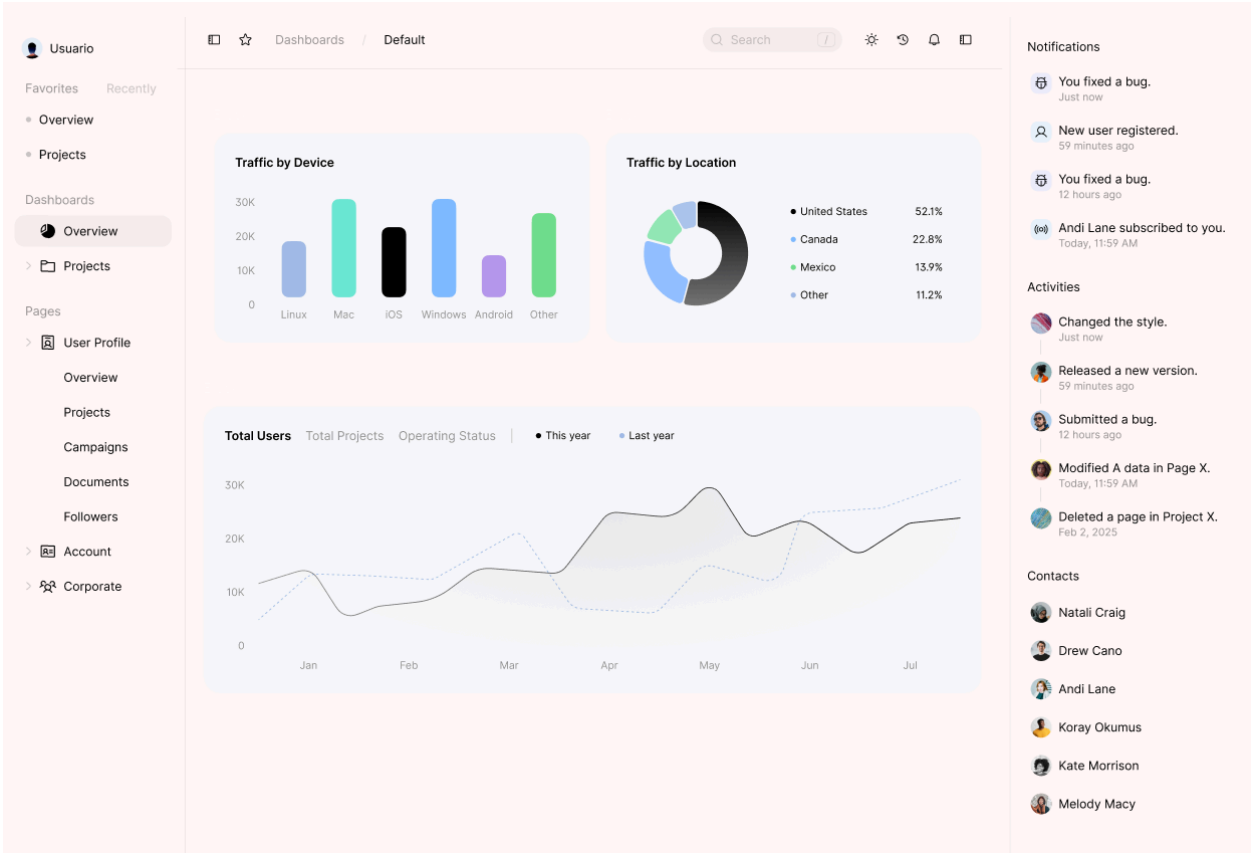
or sign up with

f

G

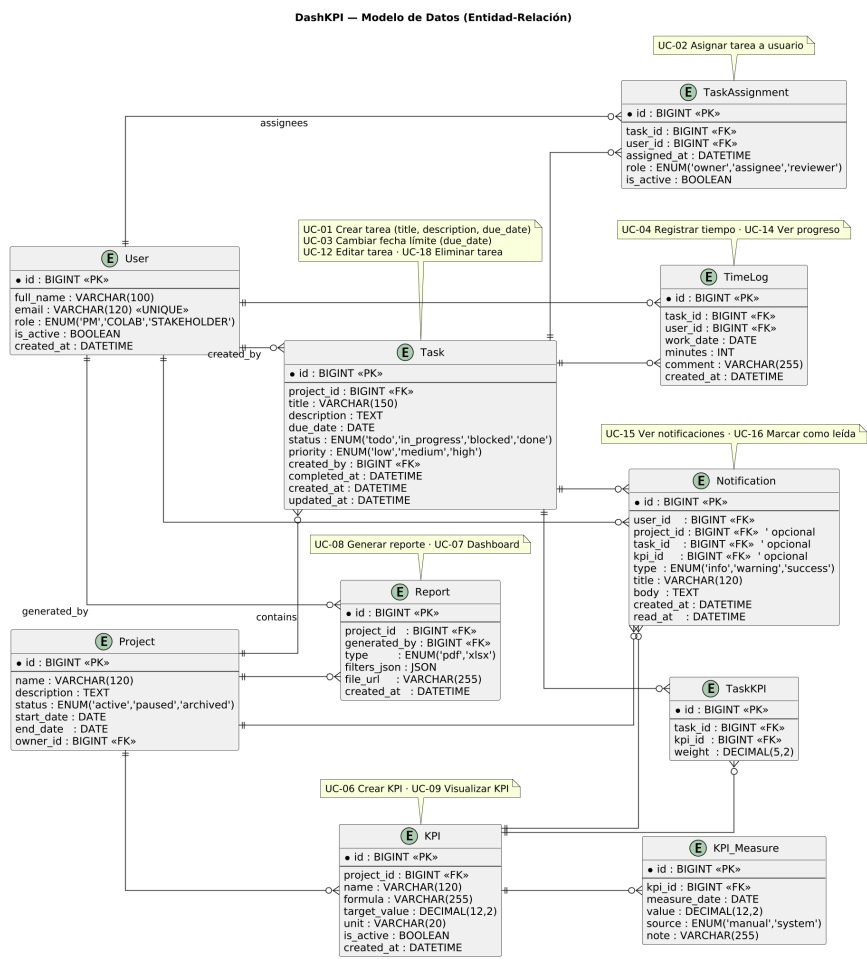
🍏

	ESPECIFICACIÓN DE DISEÑO DE SOFTWARE (SDS)	
Universidad Piloto de Colombia	PROYECTO: DASHKPI'S	Grupo:Print(Null);
		Ciclo:1



	ESPECIFICACIÓN DE DISEÑO DE SOFTWARE (SDS)	
Universidad Piloto de Colombia	PROYECTO: DASHKPI'S	Grupo:Print(Null);
		Ciclo: 1

Modelo de datos entidad relación



	ESPECIFICACIÓN DE DISEÑO DE SOFTWARE (SDS)	
Universidad Piloto de Colombia	PROYECTO: DASHKPI'S	Grupo:Print(Null);
		Ciclo: 1

Modelo de clases

